

# **COS30082**

## **Applied Machine Learning**



### **Lecture 2**

## **Overview of Machine Learning Algorithms**

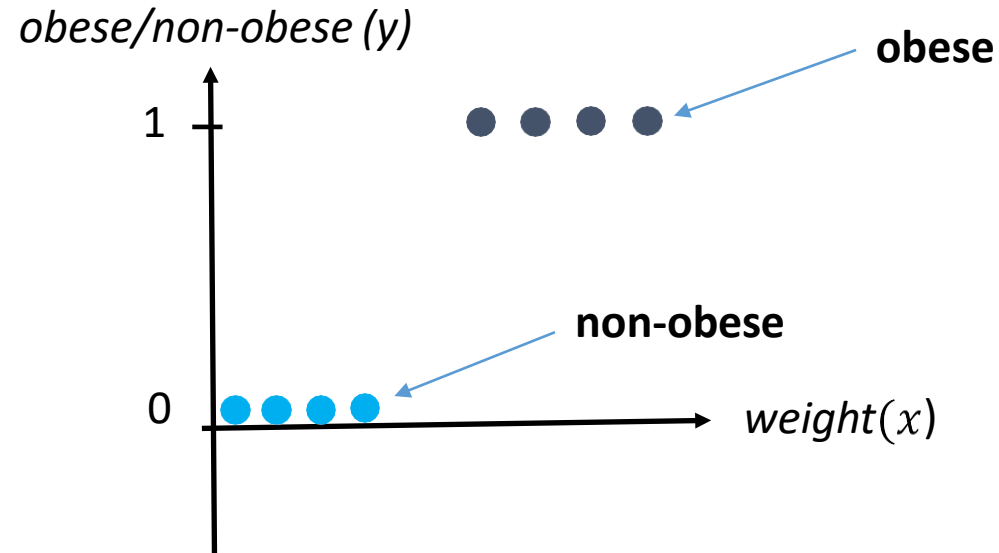
### **2.2 Logistic Regression**

# Topics



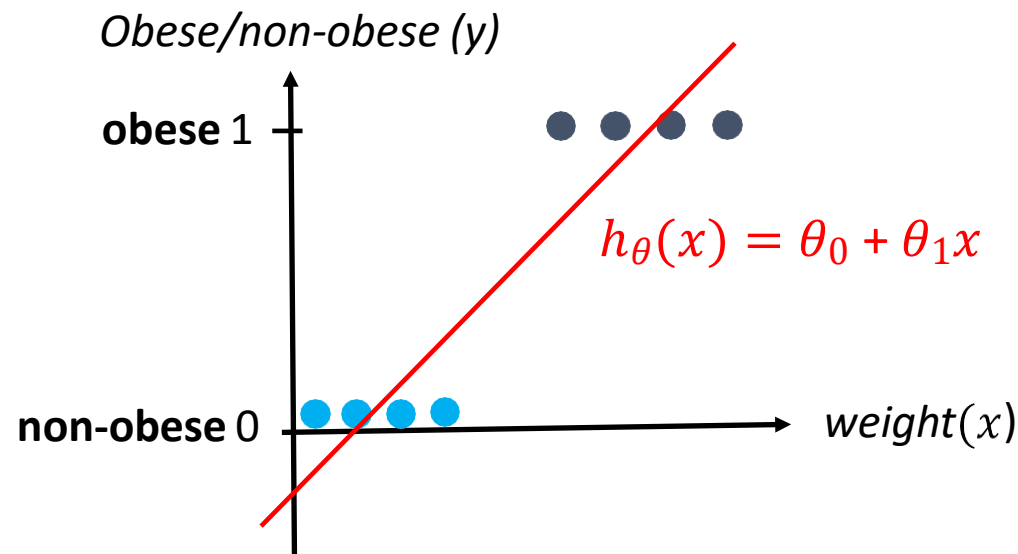
- Linear Classification Using Hard Threshold
- Logistic Regression
- Optimisation Techniques for Logistic Regression
- Multiclass Classification with Logistic Regression
- Logistic Regression in Python

# Linear classification using Hard Threshold



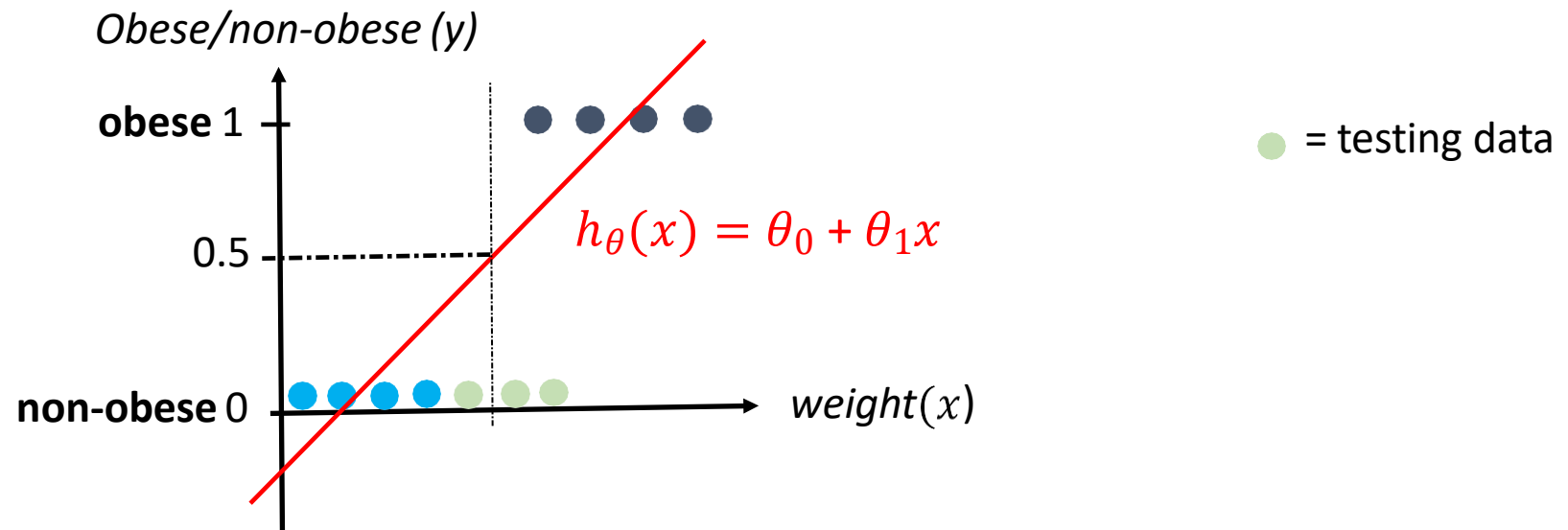
- For example, from a series of  $N$  training set, we want to model the relationship between the *obesity level* and *weight*.
- The obese patient is labeled as 1 and the non-obese patient is labeled as 0.

# Linear classification using Hard Threshold



- This red line shows the **linear regression model** that maps the independent variables (*weight*) to the dependent variable (*obesity level*).
- It learns the best fit line to minimize the distance between the predicted value  $h_{\theta}(x)$  and actual value  $y$ .

# Linear classification using Hard Threshold

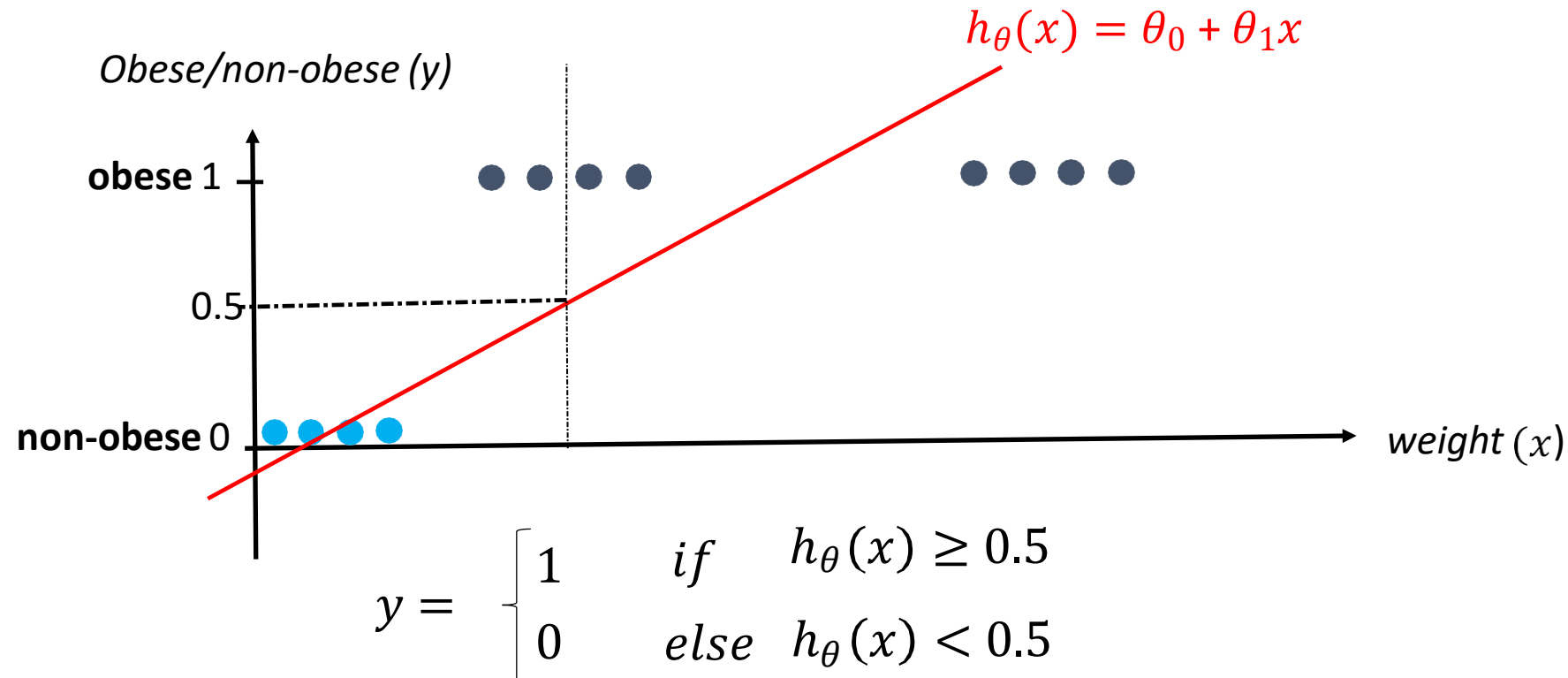


- Using a linear regression model, the predicted value  $h_{\theta}(x)$  can be classified into a real value,  $y$  (0 or 1) based on **hard threshold**.
- For example: threshold  $h_{\theta}(x)$  at 0.5

$$y = \begin{cases} 1 & \text{if } h_{\theta}(x) \geq 0.5 \\ 0 & \text{else } h_{\theta}(x) < 0.5 \end{cases}$$

# Linear classification using Hard Threshold

- However, a linear regression model used for classification is sensitive to imbalanced data and outliers.



# Linear classification using Hard Threshold

- In addition, the prediction is **continuous but not probabilistic**.
  - It is possible that  $h_{\theta}(x) < 0$  and  $h_{\theta}(x) > 1$ .
- For classification, we want to model the probability of  $y$  being **0 or 1** based on a probabilistic model,  $h_{\theta}(x) = P(y = 1|x; \theta)$ .
  - Hence, the prediction should fall within  $0 \leq h_{\theta}(x) \leq 1$ .

$$P(y=1|x; \theta)$$

- This represents the probability that the output  $y$  is equal to 1 given the input features  $x$  and the model parameters  $\theta$ .
- It indicates that the hypothesis function is a function of both the input features  $x$  and the model parameters  $\theta$ .

# Topics

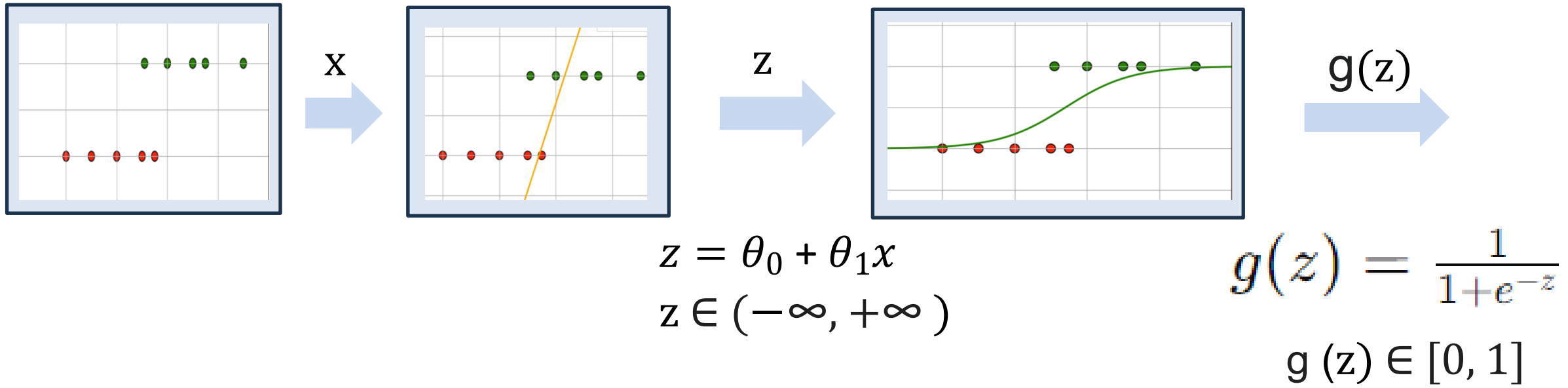
- Linear Classification Using Hard Threshold
-  • Logistic Regression
- Optimisation Techniques for Logistic Regression
- Multiclass Classification with Logistic Regression
- Logistic Regression in Python



# Logistic Regression

- **Logistic Regression** is one of the machine learning algorithms used for **classification** problems.
- It predicts the probability of a **categorical** dependent variable  $y$ , which could be represented by binary values, 0 or 1, true or false, yes or no.
- In other words, the logistic regression model predicts  $P(y = 1 | x; \theta)$  as a function of  $x$ , given parameters of  $\theta$ .

# Logistic Regression

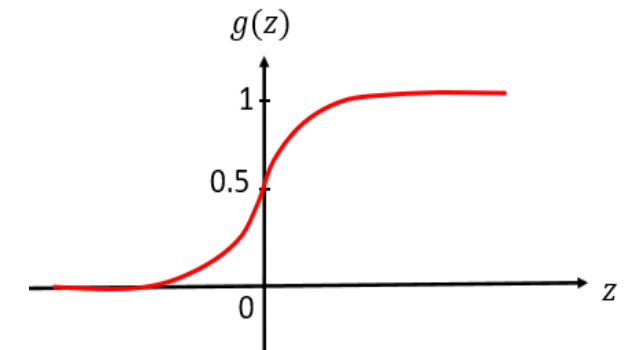


Logistic regression equation

$$h(x) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x)}}$$

Sigmoid function

$$g(z) = \frac{1}{1 + e^{-z}}$$



# Logistic Regression

- To ensure hypothesis  $h_{\theta}(x)$  lies between 0 and 1:

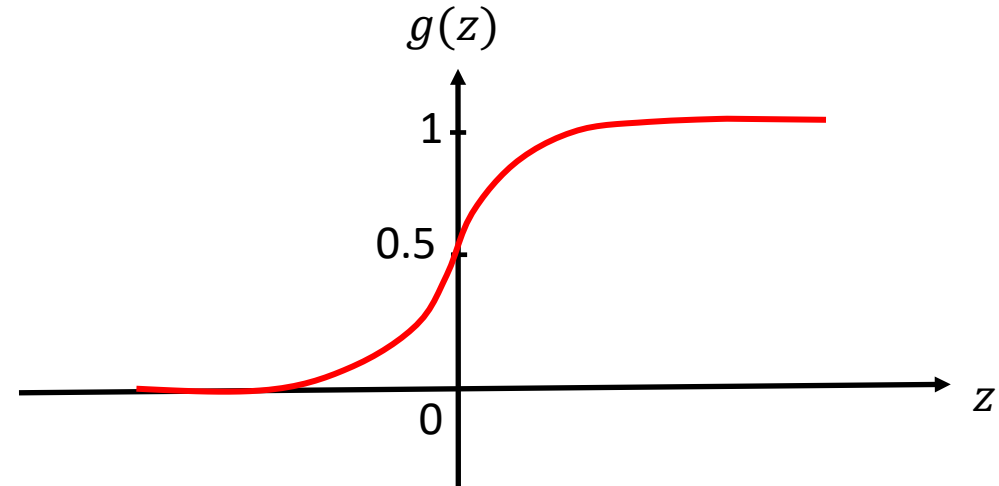
$$0 \leq h_{\theta}(x) \leq 1$$

- $z = \theta_0 + \theta_1 x$

$h_{\theta}(x)$  is represented as:

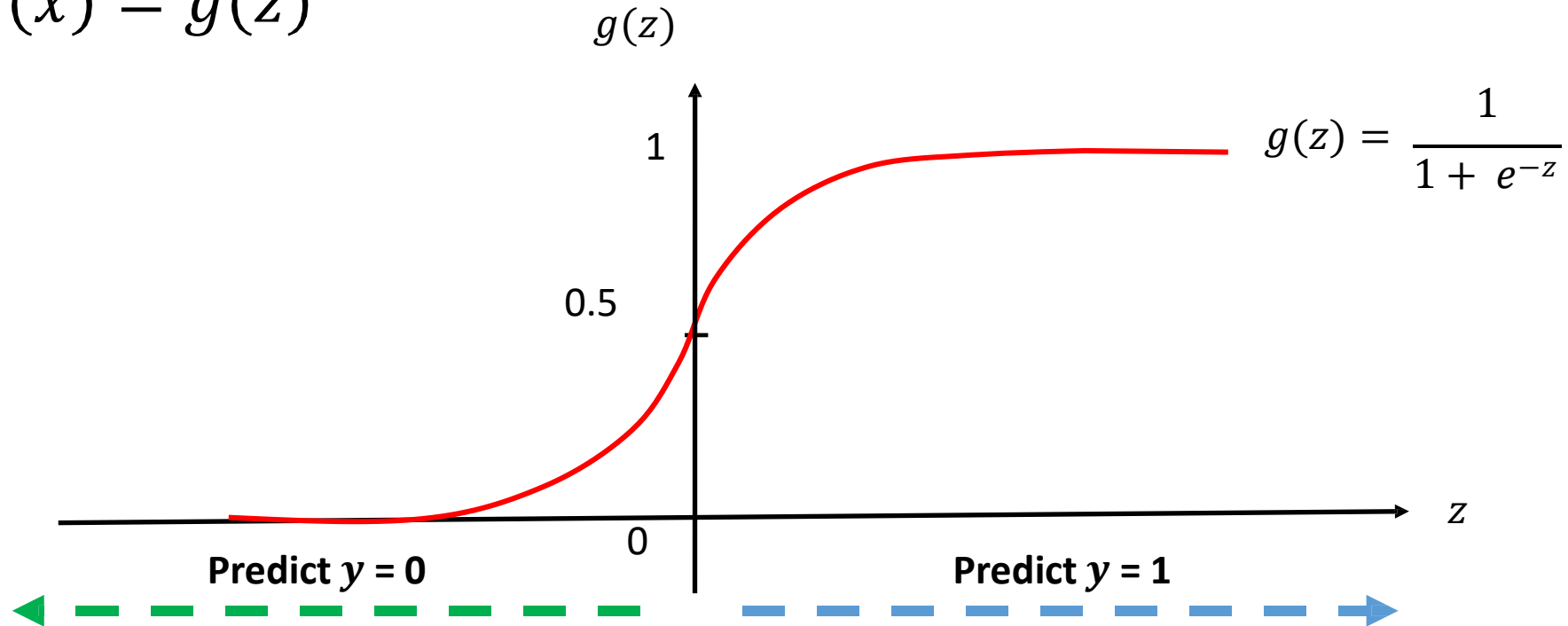
$$h_{\theta}(x) = g(z) \quad \text{where } g \text{ is a sigmoid function} \quad g(z) = \frac{1}{1+e^{-z}}$$

$$h(x) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x)}}$$



# Logistic regression

$$z = \theta_0 + \theta_1 x$$
$$h_{\theta}(x) = g(z)$$



$$z < 0$$
$$g(z) < 0.5$$

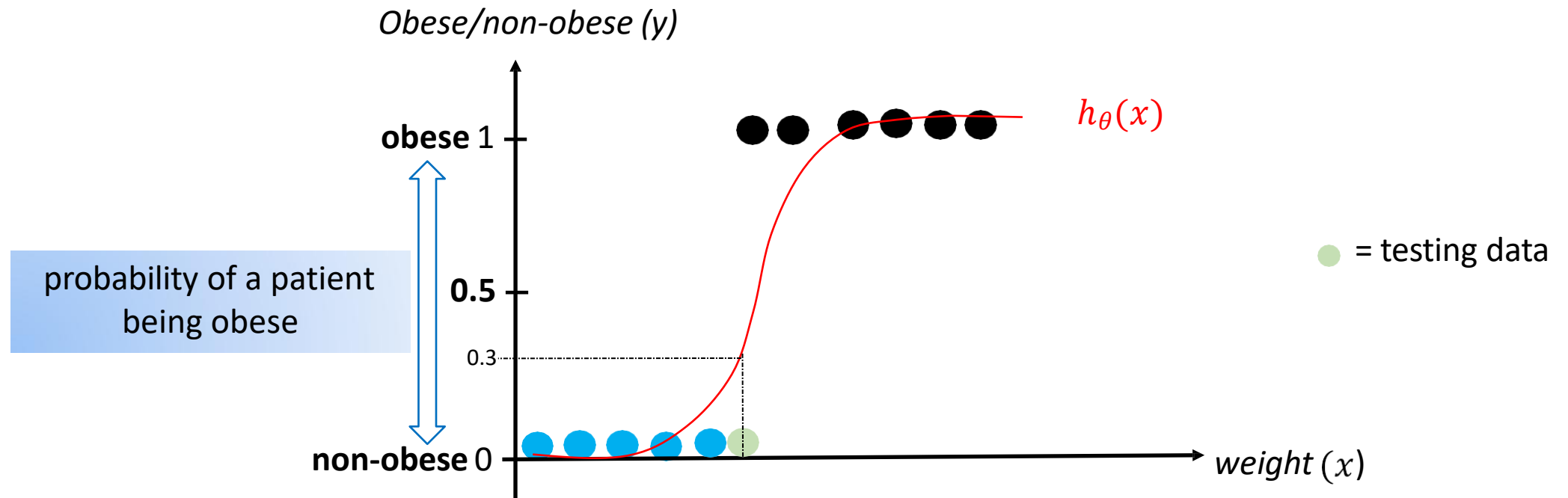
$$\theta_0 + \theta_1 x < 0$$
$$h_{\theta}(x) < 0.5$$

$$z \geq 0$$
$$g(z) \geq 0.5$$

$$\theta_0 + \theta_1 x \geq 0$$
$$h_{\theta}(x) \geq 0.5$$

# Interpretation of $h_{\theta}(x)$

- $h_{\theta}(x) = g(z) = g(\theta_0 + \theta_1 x) = g(\theta_0 + \theta_1 \cdot \text{weight})$



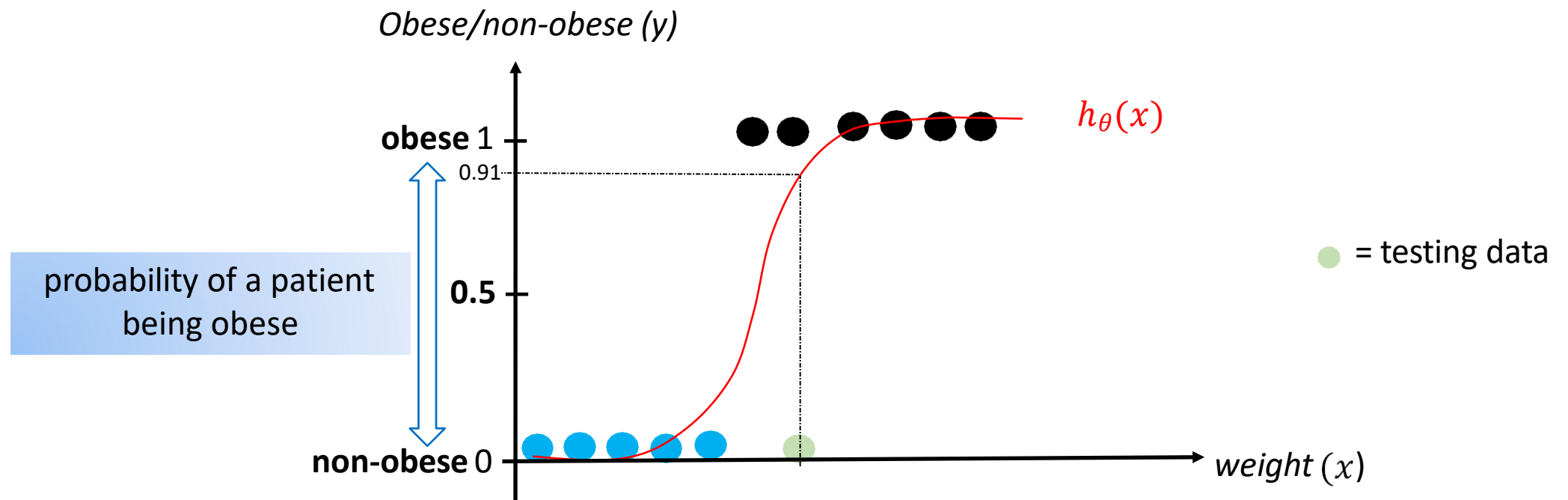
$$h_{\theta}(x) = 0.3 \longrightarrow P(y = 1 | x; \theta) = 0.3$$

Probability of 0.3 the patient is obese, given  $x$  and parameterised by  $\theta$

**Predict  $y = 0$  (non-obese)**

# Interpretation of $h_{\theta}(x)$

- $h_{\theta}(x) = g(z) = g(\theta_0 + \theta_1 x) = g(\theta_0 + \theta_1 \cdot \text{weight})$



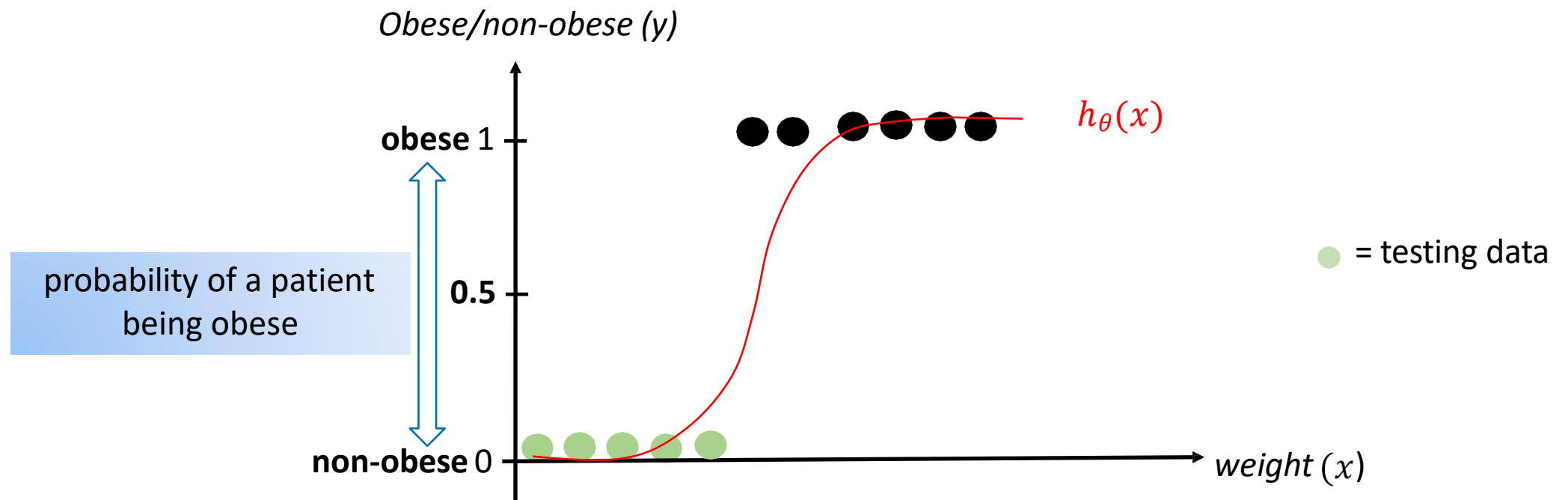
$$h_{\theta}(x) = 0.91 \longrightarrow P(y = 1 | x; \theta) = 0.91$$

Probability of 0.91 the patient is obese, given  $x$  and parameterised by  $\theta$

**Predict  $y = 1$  (obese)**

# Interpretation of $h_{\theta}(x)$

- $h_{\theta}(x) = g(z) = g(\theta_0 + \theta_1 x) = g(\theta_0 + \theta_1 \cdot \text{weight})$



$$h_{\theta}(x) = P(y = 1 | x; \theta) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x)}}$$

$$P(y = 0 | x; \theta) + P(y = 1 | x; \theta) = 1$$

$$P(\textcolor{red}{y} = \textcolor{red}{0} | x; \theta) = 1 - P(y = 1 | x; \theta)$$

# What is decision boundary?

- The **decision boundaries** separates the data-points into **decision regions**, which are actually the classes in which they belong.
- After training a machine learning model using a dataset, it is often necessary to visualize the classification of the data-points in Feature Space.



# Decision boundary

- For example, our task is to classify two different species of *Iris* [1] (*setosa* and *versicolor*) from a series of  $N$  training data, based on the features (*sepal width*, *petal length*...) of the flower organ.

setosa



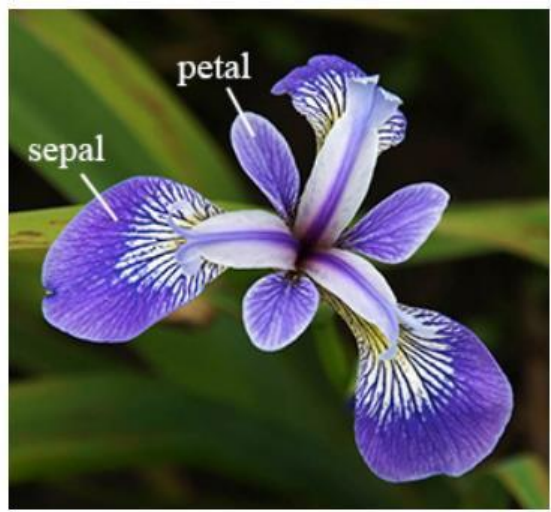
versicolor



[1] F. R. A., "The use of multiple measurements in taxonomic problems," Annual Eugenics, vol. 7, no. II, pp. 179-188, 1936

# Decision boundary

- To classify the two different species based on the *length* of the sepal:

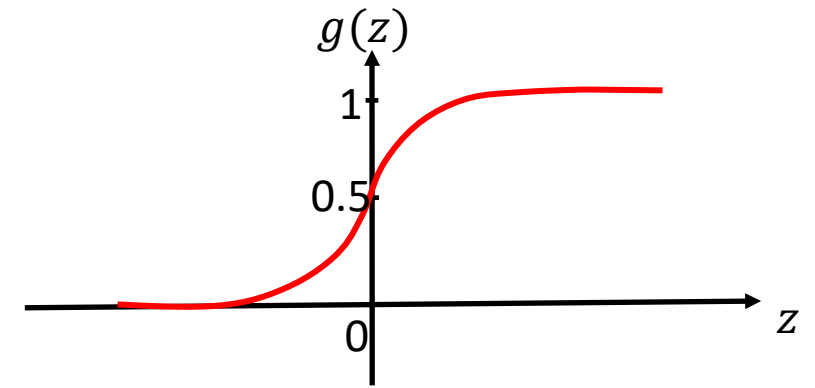
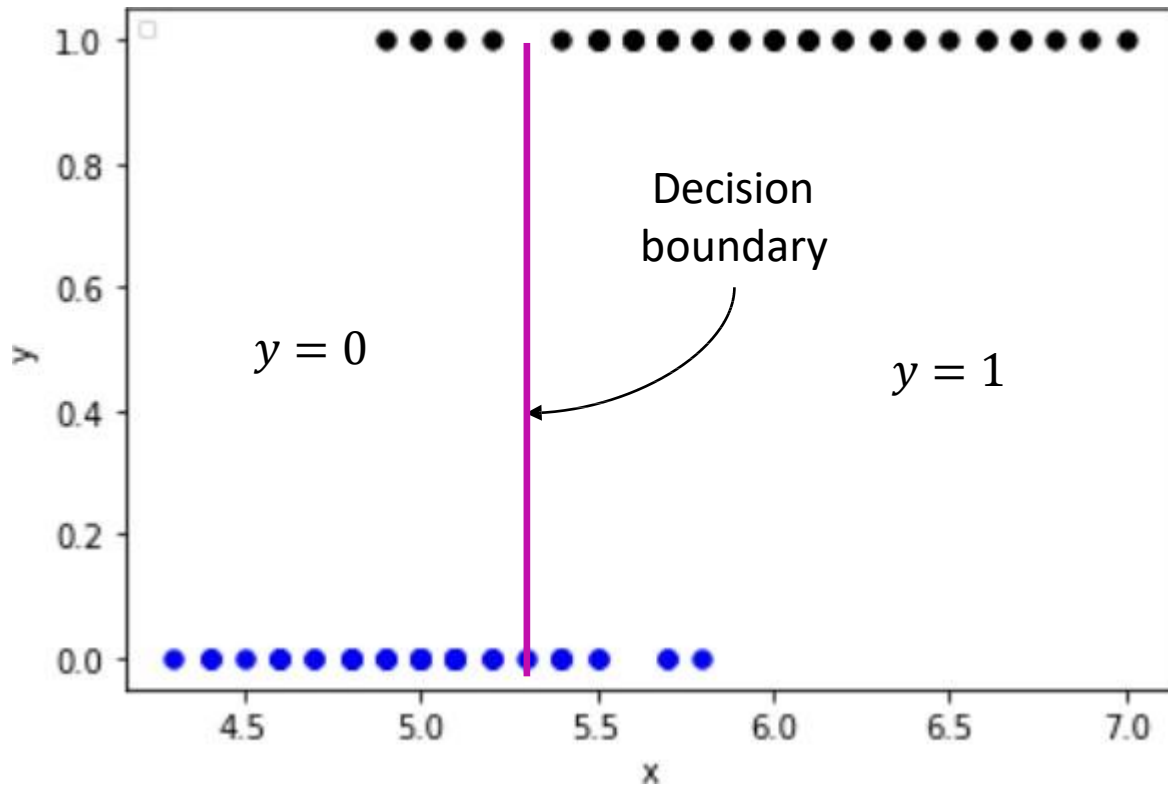


| sepal (length), $x$ | species    | $y$ |
|---------------------|------------|-----|
| 5.1                 | setosa     | 0   |
| 4.9                 | setosa     | 0   |
| 6.7                 | versicolor | 1   |
| 5.4                 | setosa     | 0   |
| 6                   | versicolor | 1   |
| ⋮                   | ⋮          | ⋮   |
| ⋮                   | ⋮          | ⋮   |

# Decision boundary

$$z = \theta_0 + \theta_1 x$$

$$h_{\theta}(x) = g(z)$$



$$y = 1 \quad \text{if } h_{\theta}(x) \geq 0.5$$

$$z \geq 0$$

$$\theta_0 + \theta_1 x \geq 0$$

$$\theta_0 = -1.033 \quad \theta_1 = 0.192$$

$$-1.033 + 0.192x \geq 0$$

$$x \geq \frac{1.033}{0.192}$$

$$x \geq 5.380$$

# Decision boundary

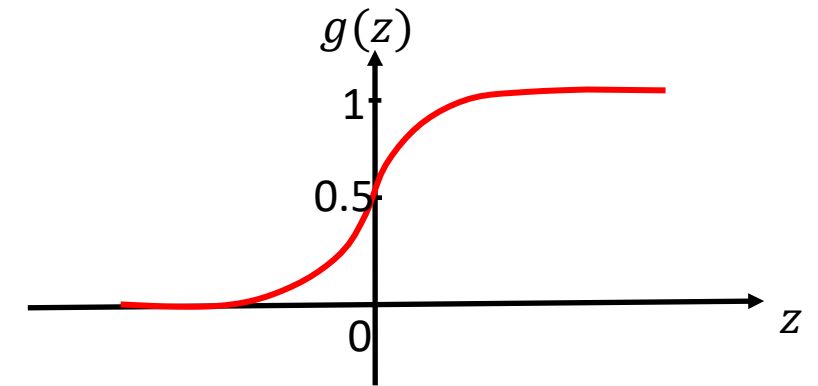
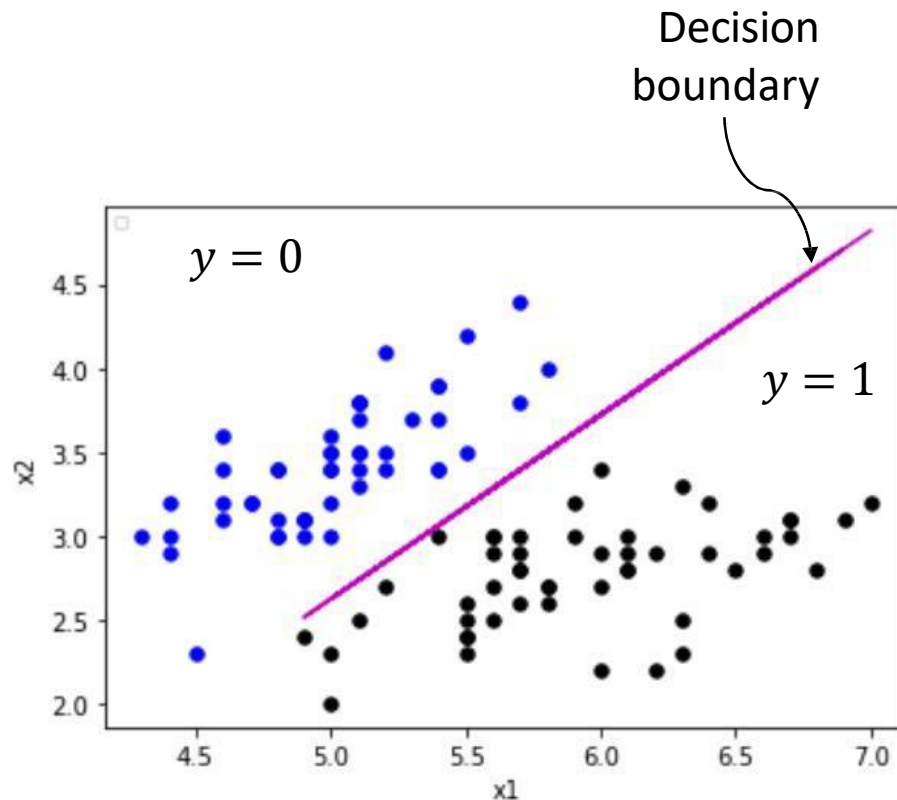
- To classify the two different species based on the *length* and *width* of the sepal:

| sepal (length), $x_1$ | sepal (width), $x_2$ | species    | $y$ |
|-----------------------|----------------------|------------|-----|
| 5.1                   | 3.5                  | setosa     | 0   |
| 4.9                   | 3                    | setosa     | 0   |
| 6.7                   | 3.1                  | versicolor | 1   |
| 5.4                   | 3.7                  | setosa     | 0   |
| 6                     | 2.2                  | versicolor | 1   |
| ⋮                     | ⋮                    | ⋮          | ⋮   |
| ⋮                     | ⋮                    | ⋮          | ⋮   |

# Decision boundary

$$z = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

$$h_{\theta}(x) = g(z)$$



$$y = 1 \quad \text{if } h_{\theta}(x) \geq 0.5$$

$$z \geq 0$$

$$\theta_0 + \theta_1 x_1 + \theta_2 x_2 \geq 0$$

$$\theta_0 = -0.588 \quad \theta_1 = 0.225 \quad \theta_2 = -0.205$$

$$-0.588 + 0.225x_1 - 0.205x_2 \geq 0$$

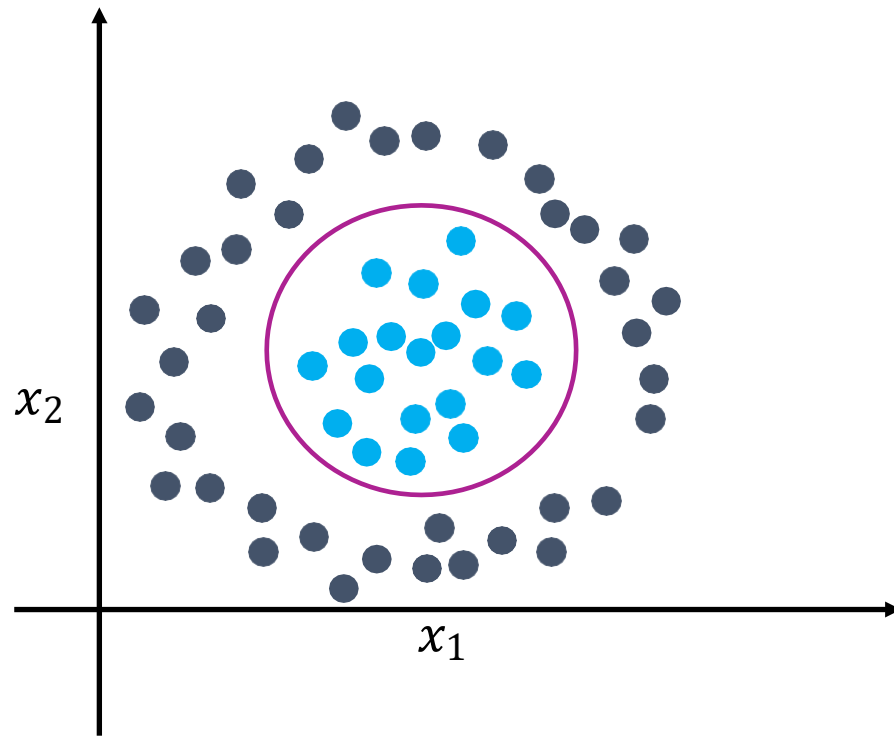
$$\frac{1}{0.205} [-0.588 + 0.225x_1] \geq x_2$$

$$x_2 \leq \frac{0.225}{0.205} x_1 - \frac{0.588}{0.205}$$

# Examples of non-linear decision boundary

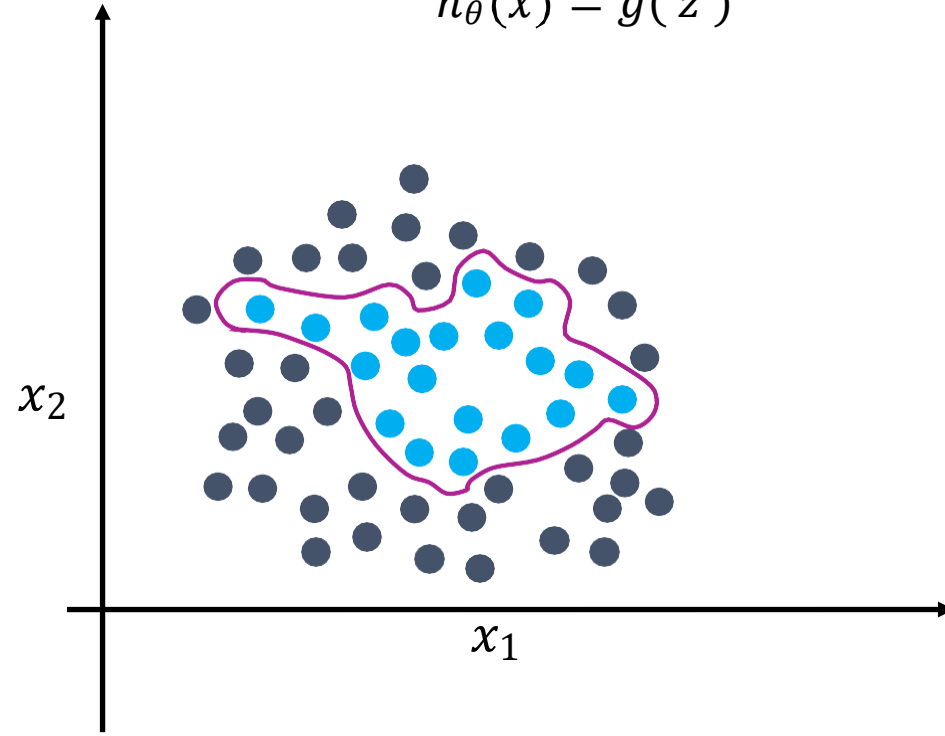
$$z = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2$$

$$h_{\theta}(x) = g(z)$$



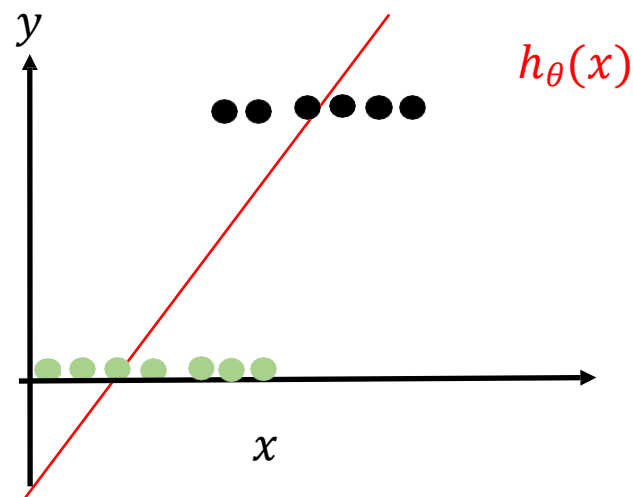
$$z = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_1^2 x_2 + \theta_5 x_1^2 x_2^2 + \dots$$

$$h_{\theta}(x) = g(z)$$



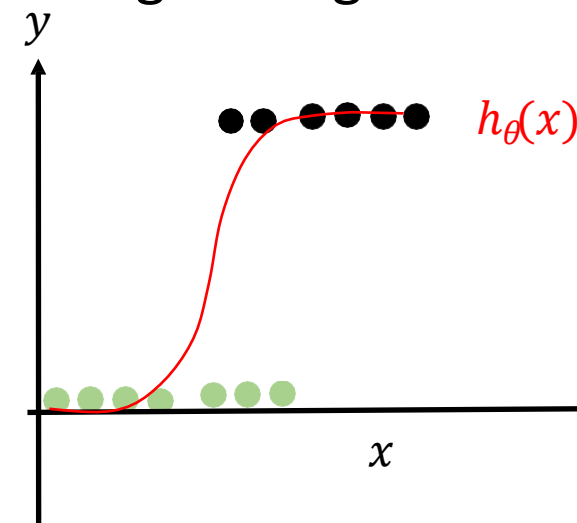
# Linear regression vs Logistic regression

## Linear regression



- Data is modelled by a **linear** function
- It is **necessary** that a linear relationship be established among dependent and independent variable
- The dependent target output,  $y$  is **continuous**

## Logistic regression



- Data is modelled by **S-shaped** logistic function
- It is **not necessary** that a linear relationship be established among dependent and independent variable
- The dependent target output,  $y$  is **discrete**

# Topics

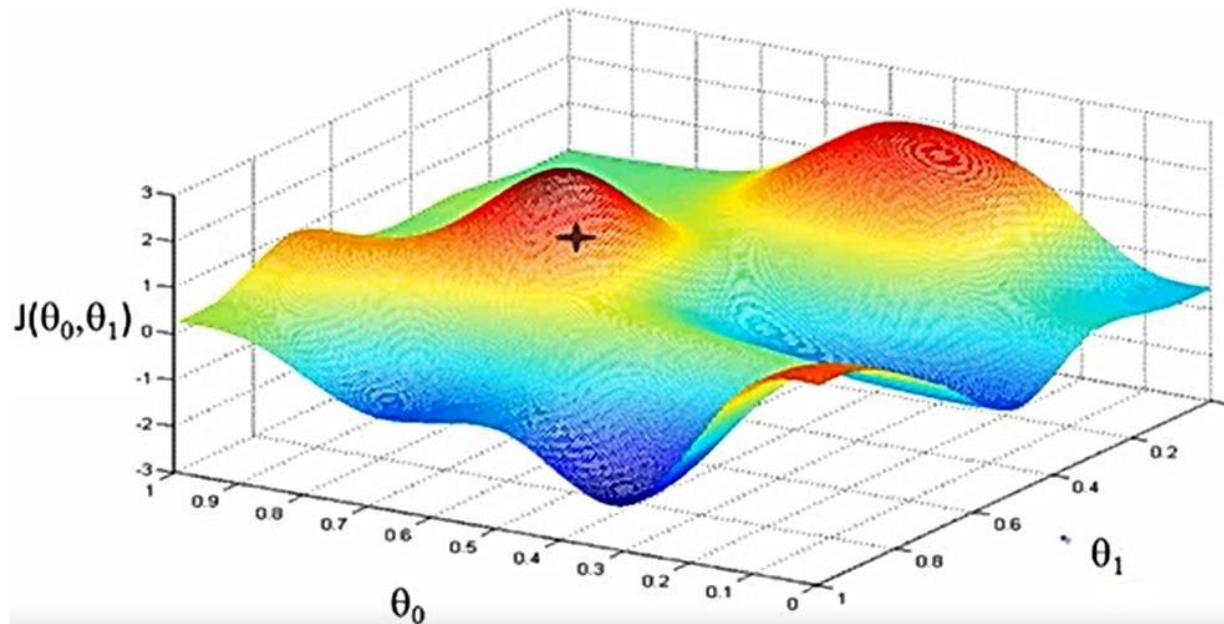
- Linear Classification Using Hard Threshold
- Logistic Regression
-  • Optimisation Techniques for Logistic Regression
- Multiclass Classification with Logistic Regression
- Logistic Regression in Python



# Cost function

$$J(\theta) = \frac{1}{2N} \sum_{n=1}^N \text{cost}(h(x^{(n)}), y^{(n)})$$

- Using linear regression cost function  $J(\theta) = \frac{1}{2N} \sum_{n=1}^N (h_{\theta}(x^{(n)}) - y^{(n)})^2$  for logistic regression may result in a **non-convex** function.
- This is due to the sigmoid function of  $h_{\theta}(x^{(n)})$  is a **non-linear** function.



# Cost function

- In logistic regression, the cost function measures the discrepancy between the predicted probabilities and the actual labels in the training data. Specifically, it quantifies how well the model's predictions align with the true outcomes.
- The binary cross-entropy loss function is commonly used as the cost function. It penalizes the model more severely for misclassifications that have higher confidence.
- By minimizing the cost function, the model adjusts its parameters to improve its predictive accuracy on the training data.

# Cost function

- The formula for the binary cross-entropy cost function is as follows:

$$J(\theta) = -\frac{1}{N} \sum_{i=1}^N [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))]$$

Where:

- $J(\vartheta)$  represents the cost function.
- $N$  is the number of training examples.
- $y^{(i)}$  is the actual label (0 or 1) of the  $i$ th training example.
- $h_{\vartheta}(x^{(i)})$  is the predicted probability that the  $i$ -th training example belongs to class 1, given by the sigmoid function applied to the input  $x^{(i)}$  and the model parameters  $\vartheta$ .

# Gradient Descent

- Hypothesis  $h_{\theta}(x)$  is represented as:

$$h_{\theta}(x) = \frac{1}{1 + e^{-(\theta^T x)}}$$

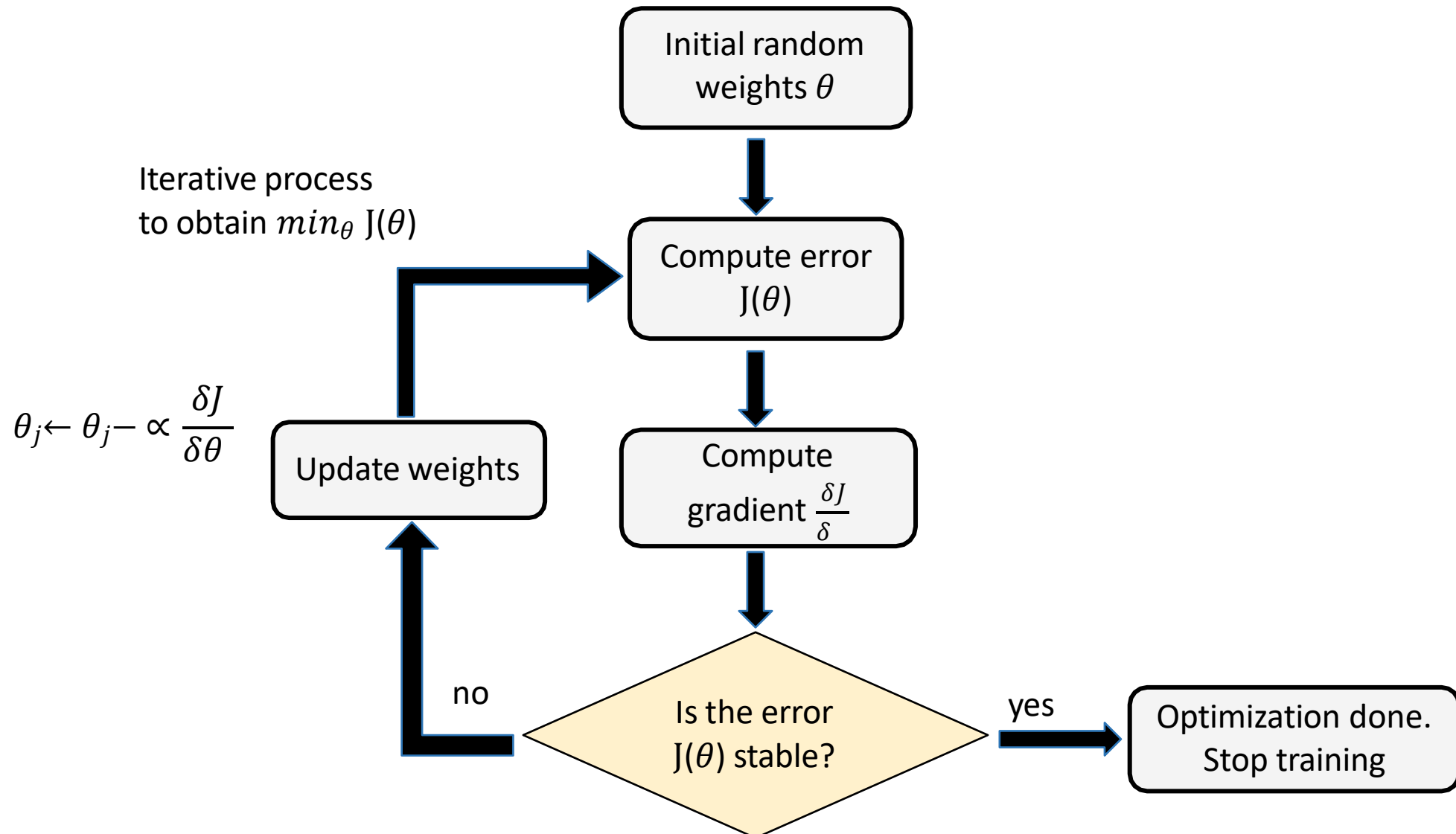
- Parameters:  $\theta_0, \dots, \theta_M$
- Cost function:

$$J(\theta) = -\frac{1}{N} \sum_{i=1}^N [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))]$$

- Goal:

$$\text{minimize}_{\theta} J(\theta)$$

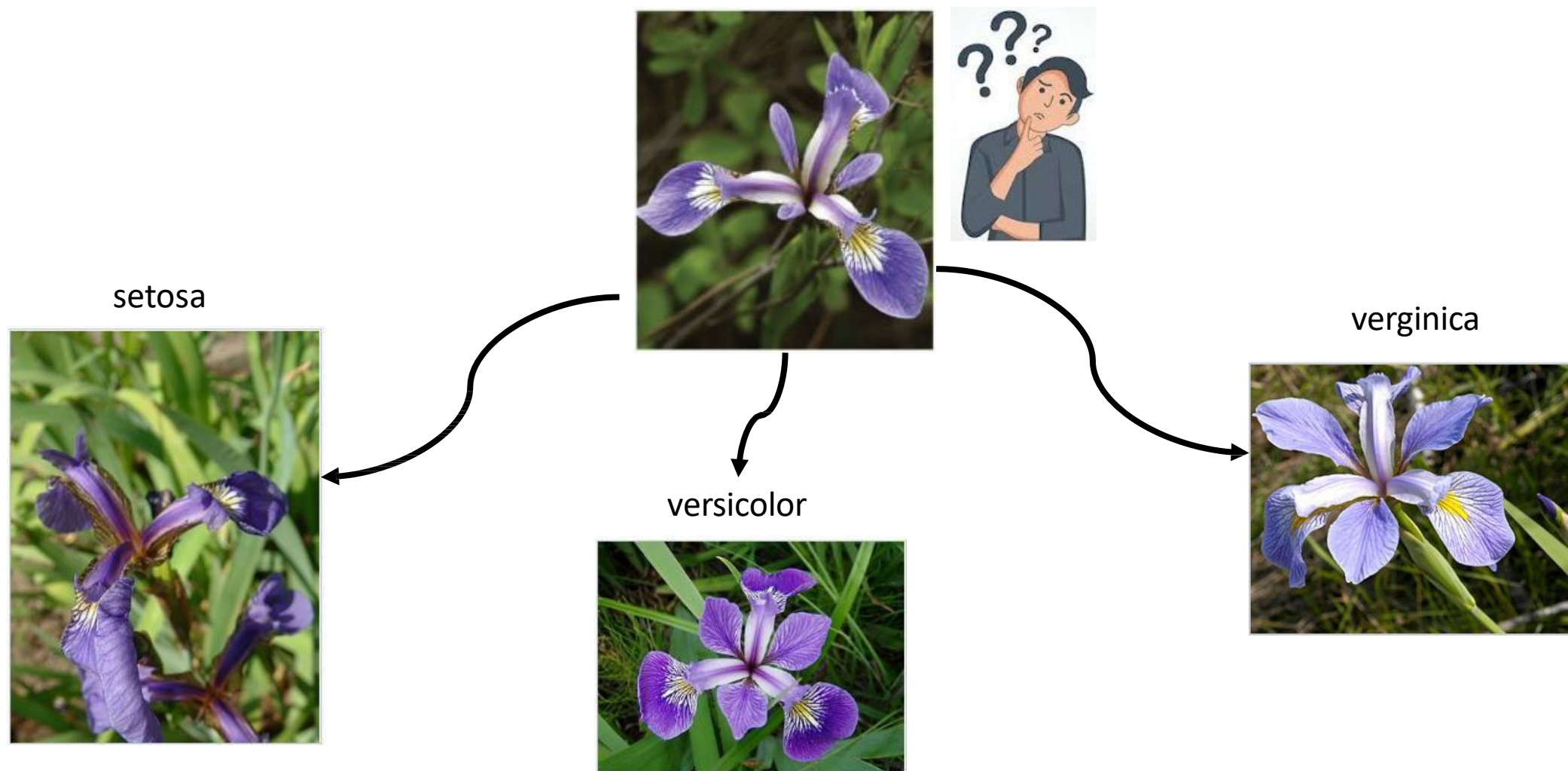
# How Gradient Descent works (revisited)



# Topics

- Linear Classification Using Hard Threshold
- Logistic Regression
- Optimisation Techniques for Logistic Regression
-  • Multiclass Classification with Logistic Regression
- Logistic Regression in Python

# Multiclass classification



# Multiclass classification

- Multiclass classification with logistic regression can be done through:
  - ➔ • one-vs-all binary logistic regression
  - multinomial logistic regression.



# One-vs-all binary logistic regression

setosa



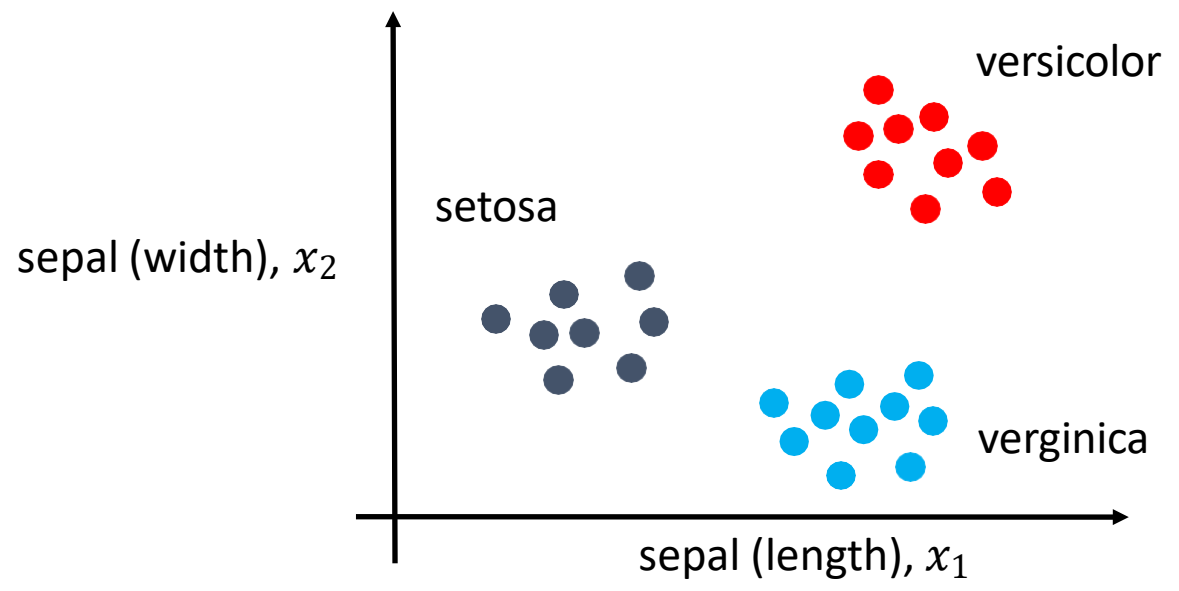
versicolor



verginica



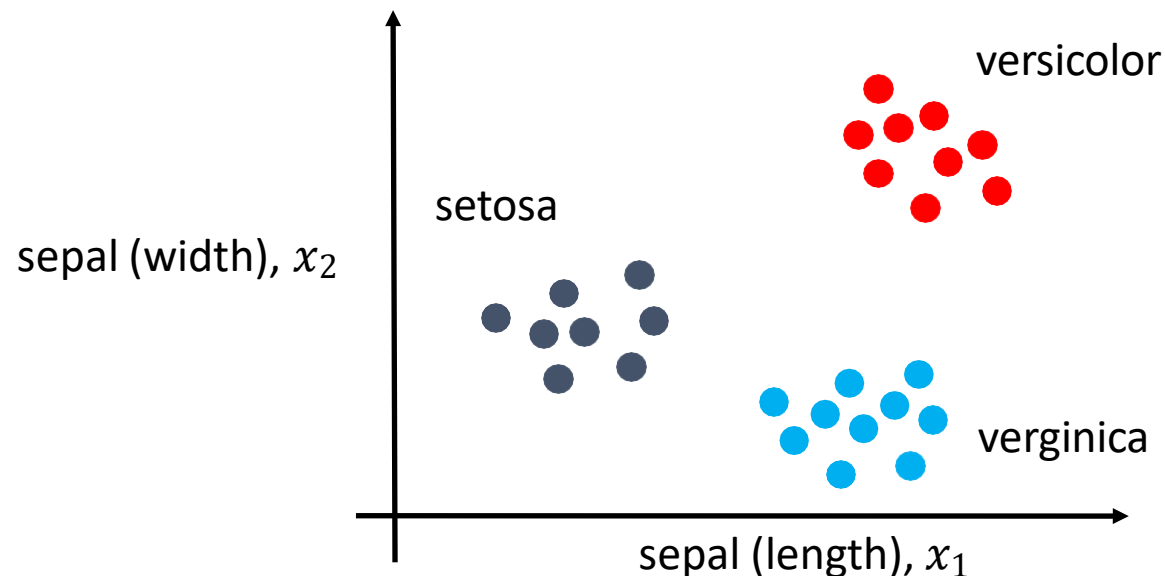
$$C = [\text{setosa} \quad \text{versicolor} \quad \text{verginica}]$$



# One-vs-all binary logistic regression

- Train a logistic regression classifier  $h^{(c)}(x)$  for each class  $c$  to predict  $P(y = c|x; \theta)$ :

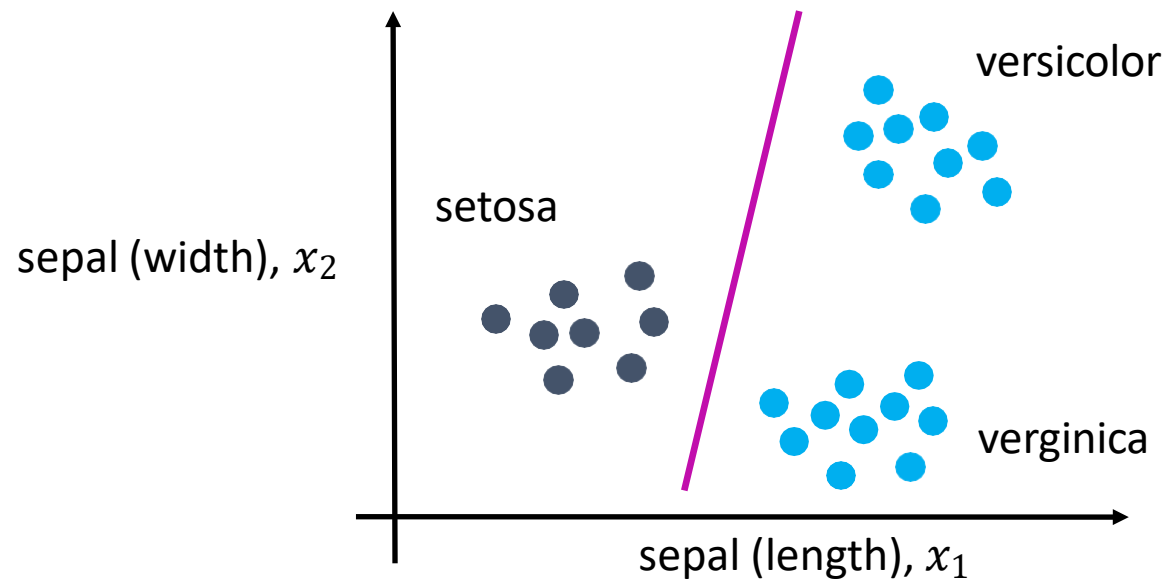
$$h_{\theta}^{(c)}(x) = P(y = c|x; \theta) \quad c \in \mathcal{C}$$



# One-vs-all binary logistic regression

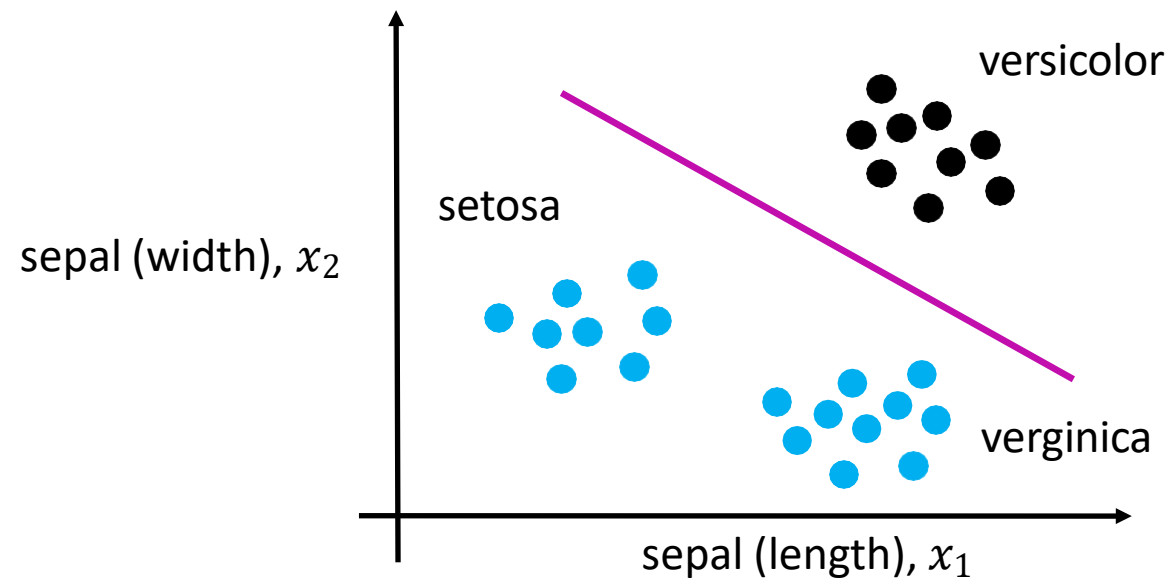
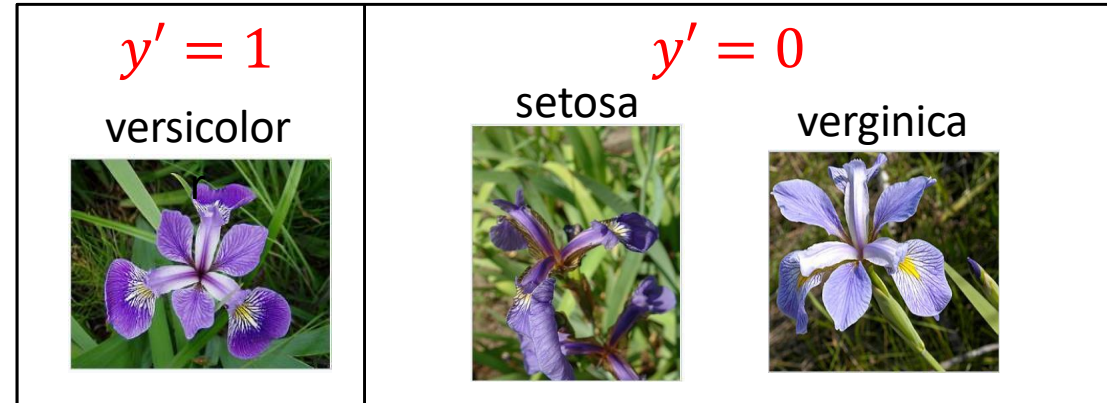
$$h_{\theta}^{(set.)}(x) = P(y = \text{setosa} | x; \theta)$$

|                                                                                                           |                                                                                                               |                                                                                                  |
|-----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|
| $y' = 1$<br>setosa<br> | $y' = 0$<br>versicolor<br> | verginica<br> |
|-----------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------|



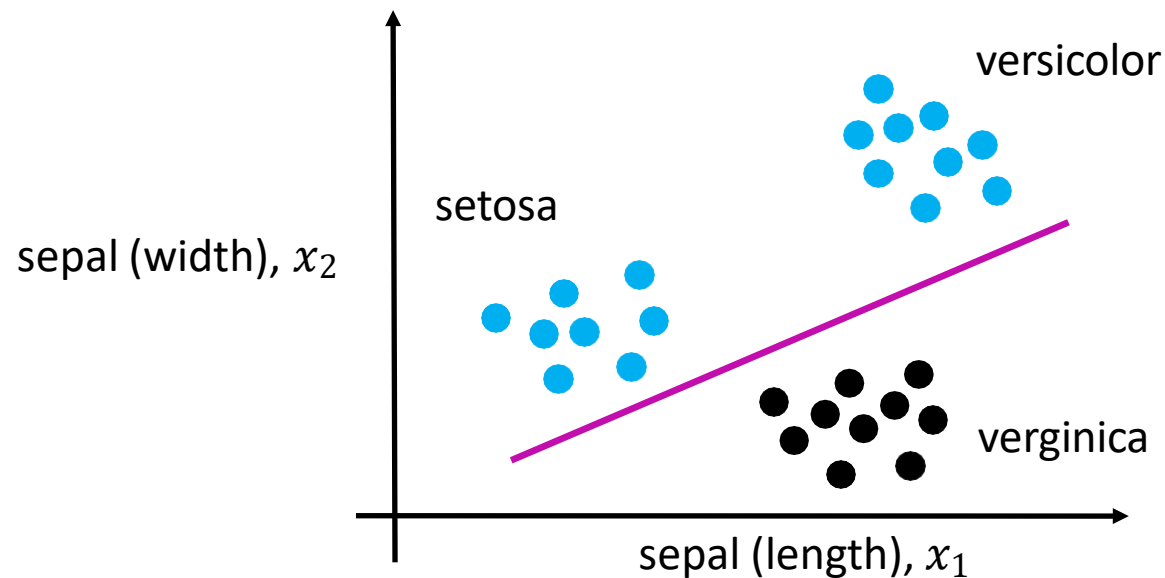
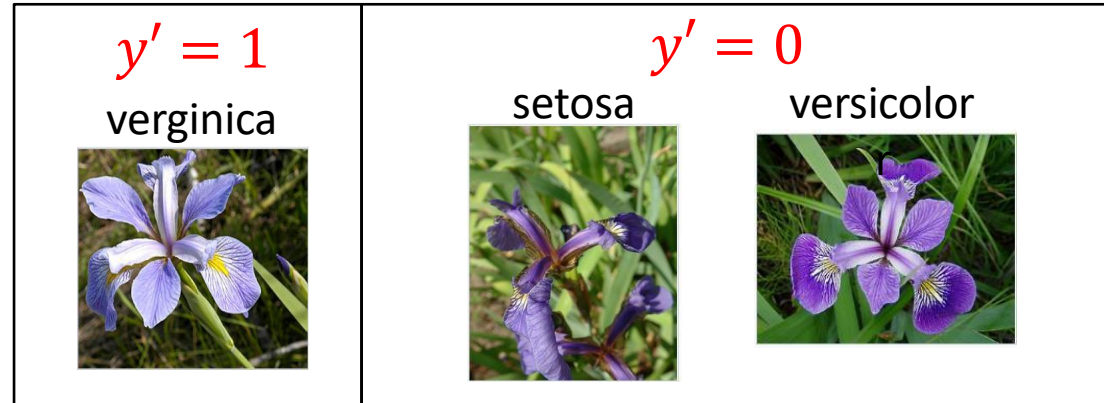
# One-vs-all binary logistic regression

$$h_{\theta}^{(vers.)}(x) = P(y = \text{versicolor} | x; \theta)$$



# One-vs-all binary logistic regression

$$h_{\theta}^{(verg.)}(x) = P(y = \text{verginica} | x; \theta)$$



# One-vs-all binary logistic regression

testing data,  $x$

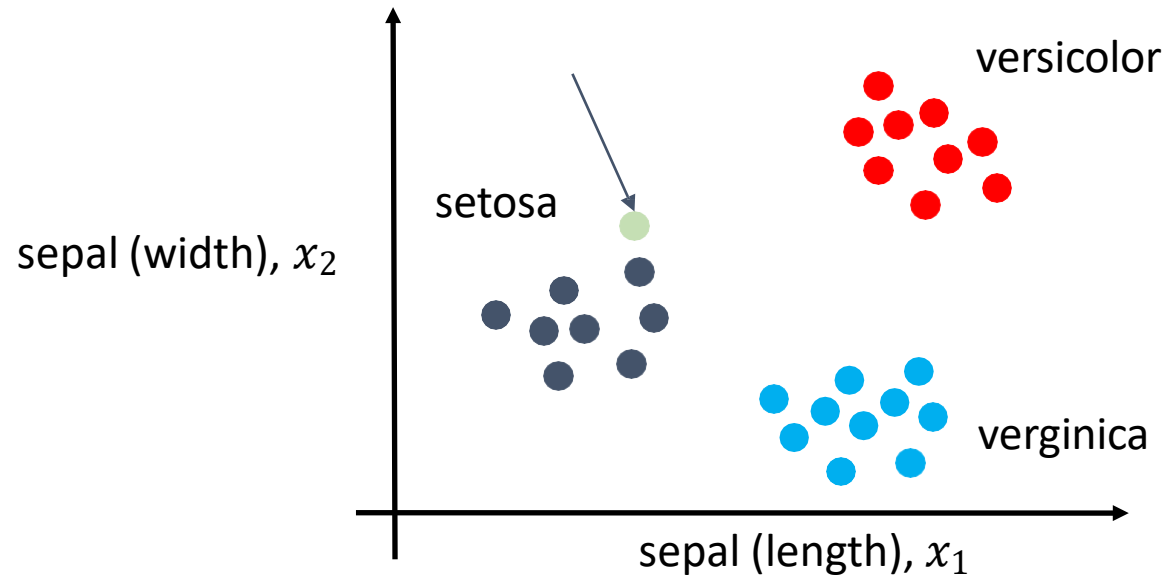


$$h_{\theta}^{(set.)}(x) = P(y = \text{setosa} | x; \theta)$$

$$h_{\theta}^{(vers.)}(x) = P(y = \text{versicolor} | x; \theta)$$

$$h_{\theta}^{(verg.)}(x) = P(y = \text{verginica} | x; \theta)$$

$$\Rightarrow \max_c h_{\theta}^{(c)}(x)$$



# Multiclass classification

- Multiclass classification with logistic regression can be done through:
  - one-vs-all binary logistic regression
  - ➔ • multinomial logistic regression.

# Multinomial logistic regression

- Multiple logistic regression, also known as multinomial logistic regression, is an extension of binary logistic regression that allows for more than two categories in the dependent variable. It's commonly used when the dependent variable has three or more categories that are unordered.
- In multiple logistic regression, the model estimates the probabilities of each category of the dependent variable and assigns observations to the category with the highest predicted probability.



# Multinomial logistic regression

- To classify the three different species based on the *length* and *width* of the sepal:

$$C = [\text{setosa} \quad \text{versicolor} \quad \text{virginica}]$$

$$y^{(n)} \in \{1, 2, 3\}$$

| sepal (length), $x_1$ | sepal (width), $x_2$ | species    | $y$ |
|-----------------------|----------------------|------------|-----|
| 5.1                   | 3.5                  | setosa     | 1   |
| 4.9                   | 3                    | setosa     | 1   |
| 6.7                   | 3.1                  | versicolor | 2   |
| 6.7                   | 3.3                  | virginica  | 3   |
| ⋮                     | ⋮                    | ⋮          | ⋮   |
| ⋮                     | ⋮                    | ⋮          | ⋮   |

# Multinomial logistic regression

$$Z = [Z_{set.}, Z_{vers.}, Z_{verg.}] = [Z_1, Z_2, Z_3]$$

$$\text{where } z_c = \theta_0^{(c)} + \theta_1^{(c)}x_1 + \theta_2^{(c)}x_2 = \theta^{(c)T}x$$

$$h_{\theta}(x) = \text{softmax}(z)$$

The softmax function is used to convert raw scores into probabilities while ensuring that the probabilities sum up to 1 across all categories. It's defined as:

$$\text{softmax}(z_c) = \frac{e^{z_c}}{\sum_{k=1}^K e^{z_k}}$$

Where:

- $\mathbf{z}$  is a vector of raw scores.  $K$  is the total number of classes
- $\text{softmax}(\mathbf{z}_c)$  is the probability of category  $c$  given the raw scores.

# Multinomial logistic regression

$$Z = [Z_{set.}, Z_{vers.}, Z_{verg.}] = [Z_1, Z_2, Z_3]$$

$$\text{where } z_c = \theta_0^{(c)} + \theta_1^{(c)}x_1 + \theta_2^{(c)}x_2 = \theta^{(c)T}x$$

$$h_{\theta}(x) = \text{softmax}(Z) = \begin{bmatrix} \frac{e^{Z_1}}{\sum_{i=1}^3 e^{Z_i}} \\ \frac{e^{Z_2}}{\sum_{i=1}^3 e^{Z_i}} \\ \frac{e^{Z_3}}{\sum_{i=1}^3 e^{Z_i}} \end{bmatrix} = \begin{bmatrix} P(y = \text{setosa} | x) \\ P(y = \text{versicolor} | x) \\ P(y = \text{virginica} | x) \end{bmatrix}$$

# Multinomial logistic regression



$C = [\text{setosa} \quad \text{versicolor} \quad \text{verginica}]$

$$h_{\theta}(x) = \text{softmax}(z) = \begin{bmatrix} 0.8 \\ 0.15 \\ 0.05 \end{bmatrix}$$

Then  $y = 1$

# Cost function

- Multinomial logistic regression has a slightly different loss function than binary logistic regression because it uses the **softmax** rather than the sigmoid classifier.
- The cost function used in multiple logistic regression is typically the cross-entropy loss function. It measures the difference between the predicted probabilities and the actual labels. The cross-entropy loss function for multiple logistic regression is given by:

$$J(\Theta) = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^K y_{ik} \log(\hat{p}_{ik})$$

Where:

- $\Theta$  represents all the parameter vectors.
- $N$  is the number of observations.
- $y_{ik}$  is an indicator variable that equals 1 if observation  $i$  belongs to category  $k$ , and 0 otherwise.
- $\hat{p}_{ik}$  is the predicted probability that observation  $i$  belongs to category  $k$ .

# Gradient Descent variation

- Gradient descent update a set of parameters in an **iterative** manner to **minimize** an error function.
- The behavior of gradient descent varies depending on the quantity of training data utilized during each iteration.
- There are three main types of gradient descent: **batch**, **stochastic**, and **mini-batch**.

# Batch Gradient Descent

- Batch gradient descent calculates the error for each example in the training dataset, but only updates the model after **all** training examples have been evaluated.
- One **training epoch** means one cycle through the **entire** training dataset.

## Advantage

- Fewer updates to the model leading to computationally efficient.
- Produces a stable error gradient and a **stable convergence**.

## Disadvantage

- A stable error gradient can sometimes result in **premature convergence** of the model to a less optimal set of parameters.
- **Resource exhausted** as it requires the entire training set resides in memory.
- Model updates, and therefore the speed of training, can become **very slow for large datasets**.

# Mini-batch Gradient Descent

- Mini-batch gradient descent splits the training dataset into **mini batches**, and calculates the error and updates the model for each mini batch.

## Advantage

- Update frequency that is higher than batch gradient descent allows for a more **robust convergence**, avoiding **local minima**.
- Mini-batch size  $<$  total training set = adding noise to the learning process, which can help to **improve the generalization** error.
- **Resource efficiency**

## Disadvantage

- Additional **hyperparameter** which is the mini-batch size for the learning algorithm.



# Stochastic Gradient Descent

- Stochastic gradient descent, often abbreviated SGD, calculates the error and updates the model for **each** example in the training dataset.

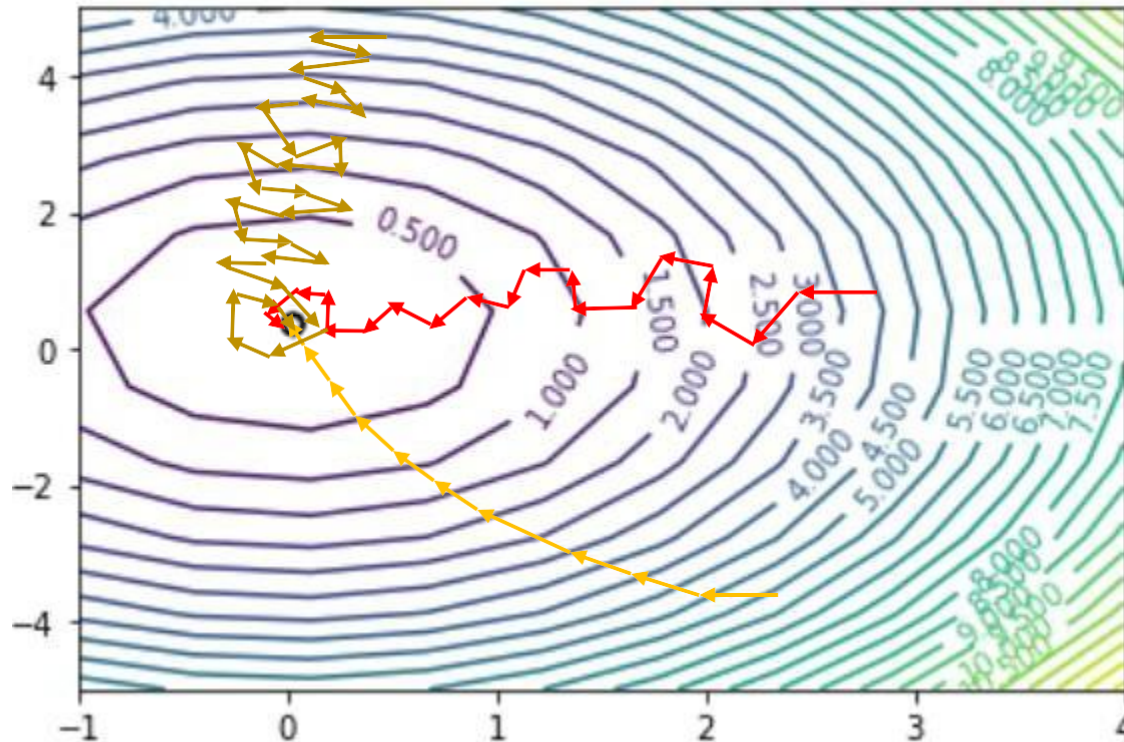
## Advantage

- Adding more noise to the learning process than mini- batch helps **to improve generalization error**.
- Faster updates can result in faster learning on some problems.
- **Resource efficiency**

## Disadvantage

- Frequent updates is more **computationally expensive** than the other gradient descent configurations.
- The noisy learning process causes the model **harder to converge**.

# Gradient Descent variation



- ← Batch Gradient Descent
- ← Mini-batch Gradient Descent
- ← Stochastic Gradient Descent

- Mini-batch Gradient Descent which uses a combination of Stochastic Gradient Descent and Batch Gradient Descent is often a preferred method to be used.

# Topics

- Linear Classification Using Hard Threshold
- Logistic Regression
- Optimisation Techniques for Logistic Regression
- Multiclass Classification with Logistic Regression
- Logistic Regression in Python



# Logistic Regression in Python

## Python libraries

- Numpy: library for efficient numerical operations.
- Pandas: library for data manipulation.
- Scikit-learn: library of machine learning tools.
  - `linear_model`: Implements logistic regression and other linear models.
  - `svm`: Module for support vector machine algorithms.
  - `tree`: Module for decision tree-based algorithms.

## Import libraries

```
import numpy as np
import pandas as pd
from sklearn import linear_model
```

# Logistic Regression in Python

Split training and testing data

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, Y_train, Y_test = train_test_split(x, y, test_size=0.2,  
random_state=0)
```

**test\_size:** specifies the proportion of the dataset that should be allocated for testing.

**random\_state:** controls the randomness of the data splitting process.

When `random_state` is set to an integer, it acts as a seed, ensuring that the data splitting process is reproducible.

# Logistic Regression in Python

Fit (train) the Logistic Regression classifier

```
clf = linear_model.LogisticRegression(C=1e40, solver='newton-cg')  
fitted_model = clf.fit(X_train, Y_train)
```

- **C:** The C parameter represents the inverse of regularization strength. Regularization is a technique used to prevent overfitting by penalizing large coefficients. A smaller value of C implies stronger regularization, where the model is penalized more for large coefficients. Default 1.0.
- **solver:** The solver parameter specifies the optimization algorithm to use in the logistic regression model.

# Logistic Regression in Python

Evaluate model:

```
from sklearn.metrics import accuracy_score, precision_score, recall_score,  
f1_score
```

```
# Predict on the test data
```

```
y_pred = clf.predict(X_test)
```

```
# Calculate evaluation scores
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
precision = precision_score(y_test, y_pred)
```

```
recall = recall_score(y_test, y_pred)
```

```
f1 = f1_score(y_test, y_pred)
```

```
# Print the evaluation scores
```

```
print(f"Accuracy: {accuracy}")
```

```
print(f"Precision: {precision}")
```

```
....
```

# Logistic Regression in Python

Make predictions:

```
data = pd.DataFrame([[1, 2, 48],[2, 0, 45]],  
                    columns=['feature1', 'feature2', 'feature3'])
```

```
# Make predictions on new data
```

```
predictions = clf.predict(data)
```

```
# Print the predicted class labels
```

```
print(predictions)
```

The prediction is an array containing all the predicted class labels for the new data.



# Next Topic

❖ Support vector machine (SVM)